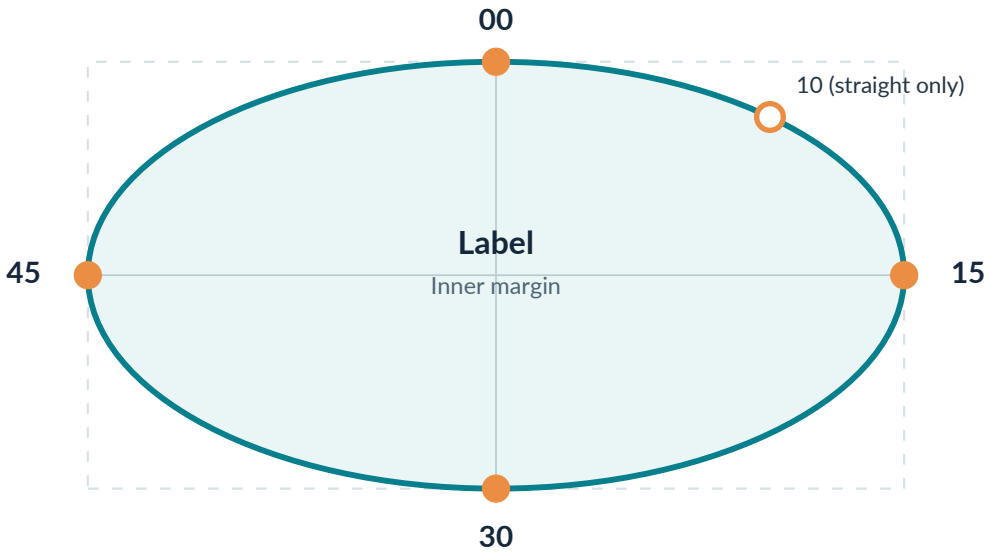
IMPLEMENTATION SPECIFICATION / V3

Elliptical nodes

A shape-aware plain node for **svg-dom-graph**

> **V3 resolution:** Each edge end resolves from its own geometry and bearing. Automatic straight edges intersect continuously; automatic elbows use angular cardinal positions. Explicit off-cardinal clocks are straight-only. New geometry and error contracts are fixed.



Dotted box = axis-aligned bounds used by existing scene geometry

Purpose. Add circular and elliptical nodes without changing rectangular node semantics, the current four-point route buffer, or the graph ownership model.

Status. Proposal only; API examples are proposed, not implemented. Checked against ChrisWhealy/svg-dom-graph main and the svg-dom ellipse factory on 23 October 2026.

01 / Normative v1 decisions

The following rules are **required behaviour**, including at both connector ends. An ellipse is an axis-aligned shape with horizontal/vertical radii, with a circle as the equal-radii case.

Case	Required behaviour
Automatic straight	For each ellipse endpoint, intersect the ray towards the **other centre** with the actual curved outline. Never snap automatically.
Automatic elbow	For each ellipse endpoint, choose the nearest cardinal by angular bearing: 00=N, 15=E, 30=S, 45=W. Diagonal ties resolve clockwise.
Explicit clock / straight	ClockPosition 0..=59 is allowed; the given minute fixes that endpoint on the ellipse outline, regardless of the other centre.
Explicit clock / elbow	Only cardinals 00, 15, 30 and 45 are allowed. Off-cardinals return Error::InvalidEllipseElbowClock.
Forced side on ellipse	For **both straight and elbow**, N/E/S/W forces 00/15/30/45. Thus a forced-side straight edge is no longer a continuous-ray crossing.
Mutable clock position	No set_edge_clock in v1. Clocks are saved from add_edge_with; changing type still checks saved clocks.
Undefined direction	Coincident centres: each ellipse endpoint uses 15 (east). No division by zero. Rectangle fallback behaviour remains unchanged.

Shape-specific behaviour

- Each **from** ellipse uses the bearing *from centre to to centre*; each **to** ellipse uses the reverse bearing *from to centre to from centre*.
- The rectangle end of a mixed edge still uses the existing rectangle side, anchor and fraction rules. It need not choose a cardinal matching the ellipse end.
- Explicit pins are independent at the two ends and retain their values after movement. No anchor position has exclusive capacity.

02 / Pure geometry and clock positions

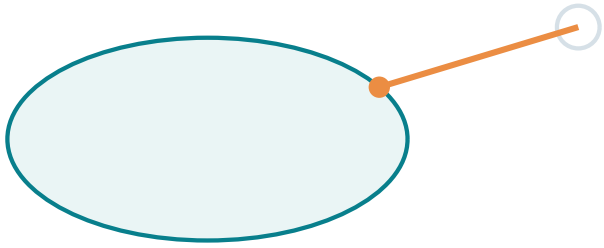
SVG coordinates increase east on x and south on y. Ellipse centre C=(cx,cy); radii rx>0, ry>0. The final bounding rectangle is (cx-rx, cy-ry, 2rx, 2ry).

```
((x-cx)/rx)^2 + ((y-cy)/ry)^2 = 1

theta = 2*pi*m/60                // m is 0..59
dx = sin(theta); dy = -cos(theta)
t  = 1 / hypot(dx/rx, dy/ry)
P(m) = (cx + t*dx, cy + t*dy)
```

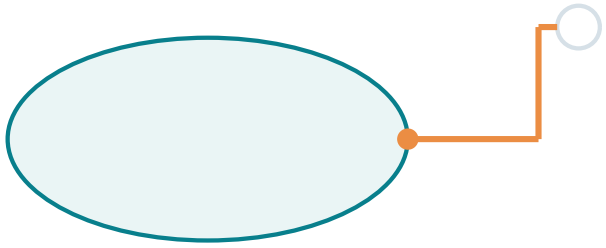
This is a **ray direction**, not the usual parametric ellipse angle. Check all input, intermediate and resulting coordinates for finiteness; avoid numerical overflow. P(m) must satisfy the ellipse equation to tolerance.

STRAIGHT: continuous



Ray intersection

ELBOW: cardinal



East fixing point (15)

Automatic endpoints, independently resolved

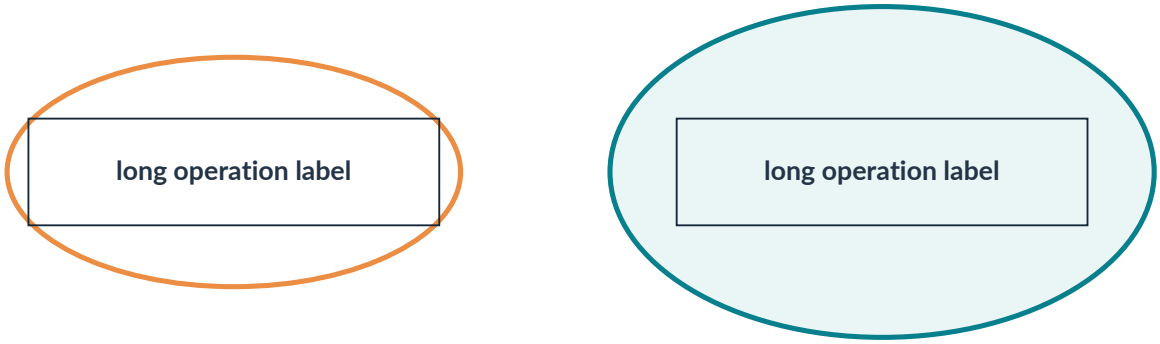
```
from_dir = to.centre   - from.centre
to_dir   = from.centre - to.centre
// Straight ellipse: ray_intersection(ellipse, its_dir)
// Elbow ellipse:   nearest_cardinal(its_dir)
// atan2(dir.x, -dir.y) yields clockwise bearing from N
// Tie at 45/135/225/315 deg: next clockwise cardinal
```

For a wide ellipse (rx=100, ry=40), rays at 30°/40° *above the positive x-axis* select E; rays at 50°/60° select N. This intentionally differs from the rectangle exit-side threshold atan(40/100)=21.8°. Rectangle ends continue using their original side logic.

The output cardinal also supplies its outward N/E/S/W side to the existing elbow router. Mixed-side pairs are legal (for example E at an ellipse end and N at a rectangle end).

03 / Label fitting and dimensions

Unlike ordinary plain rectangles, ellipses **preserve normal label font size** and enlarge uniformly if the label and inner margin would cross their curved boundary. No shrink-to-fit policy in v1.



Insufficient Expanded

Curve-aware containment

Measure the actual SVG text getBBox in a measurable context; centre the *visual* bounding box, accounting for its offset. For text width w, height h and inner margin M:

```
hx = w/2 + M;   hy = h/2 + M
q = hypot(hx/rx, hy/ry)
s = max(1, q)
rx_final = s*rx; ry_final = s*ry
// Within tolerance: (hx/rx_final)^2 + (hy/ry_final)^2 <= 1
```

This encloses all four corners of the padded label rectangle. Scaling both radii by the same s preserves aspect ratio, so circles remain circles. Centre stays fixed during growth. Reject nonfinite measured metrics, scale, radii and final Rect.

Default M = **6 SVG user-space units** (configurable, finite, non-negative). With an 80x16 label, circle radius 30 grows to ~48.1. Ellipse radii 70x32 already fit. A 10x16 glyph at radius 20 fits with M=6; M=8 would require radius ~20.6.

Measured input	Expected rule
Width 0, nonzero height	Finite calculation; no division by text size.
Empty label / 0x0 bbox	Margin alone is considered. Radii remain positive; no NaN.
Demo circular vertices	Start with radius 22 or more to leave room for letters and focus stroke.

No post-construction label/radius setters or silent font-change reflow in v1. A later refresh API would have to recompute bounds, incident paths and edge labels.

04 / Public API and coordinate conventions

Dedicated constructors keep NodeOptions and EdgeAnchors(n) untouched: for rectangular nodes EdgeAnchors means n candidates *per side*. Proposed types use private fields and remain Copy where practical.

```
#[derive(Debug, Clone, Copy, PartialEq)]
pub struct EllipseRadii { rx: f64, ry: f64 }
impl EllipseRadii {
    pub fn circle(radius: f64) -> Self;
    pub fn new(rx: f64, ry: f64) -> Self;
    pub fn rx(&self) -> f64;
    pub fn ry(&self) -> f64;
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct ClockPosition(u8);
impl ClockPosition {
    pub const NORTH: Self = Self(0);
    pub const EAST:  Self = Self(15);
    pub const SOUTH: Self = Self(30);
    pub const WEST:  Self = Self(45);
    pub fn minute(self) -> u8;
}
impl TryFrom<u8> for ClockPosition {
    type Error = ClockPositionError;
    // Reject 60..=255; never wrap modulo 60.
}

#[non_exhaustive]
#[derive(Debug, Clone, Copy, PartialEq)]
pub struct EllipseOptions { /* private fields */ }
// Default fill = plain-node fill; inner_margin = 6.0
// with_fill(Option<&'static str>), with_inner_margin(f64)
```

Public entry points

```
Scene::add_ellipse_node(centre, radii, label) -> Result<NodeId, Error>
Scene::add_ellipse_node_with(centre, radii, label, options)
    -> Result<NodeId, Error>
ConnectorOptions::with_from_clock(Option<ClockPosition>) -> Self
ConnectorOptions::with_to_clock(Option<ClockPosition>) -> Self
```

Coordinate contract. Both ellipse add methods accept the **centre**, while move_node continues to take the **top-left of the actual, final enclosing Rect**. node_rect returns that final Rect; label growth keeps the supplied centre fixed. Repeat the same explanation in constructor and move_node documentation.

EllipseRadii builders deliberately do not validate; add_ellipse_node(_with) validates all geometry and margins before inserting a model node. Private fields make EllipseRadii non-exhaustive in practice without requiring a redundant attribute.

05 / Exact validation and error contract

These are fixed public error names and payloads for v1; do not substitute "or equivalent". They augment the existing non_exhaustive graph Error enum.

```
// Variants of svg_dom_graph::Error:
InvalidEllipseGeometry { centre: Point, rx: f64, ry: f64 },
InvalidEllipseMargin(f64),
InvalidEllipseEndpoint {
    endpoint: &'static str, // "from" or "to"
    reason: &'static str,   // documented reason
},
InvalidEllipseElbowClock(ClockPosition),

// Separate, public, zero-dependency conversion error:
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct ClockPositionError { value: u8 }
// fn value(&self) -> u8; impl Display + std::error::Error
// ClockPosition::try_from(60u8) -> Err(ClockPositionError { ... })
```

Trigger	Required error and no-mutation rule
Invalid centre, rx/ry, bounding Rect, fitted radii or overflow	InvalidEllipseGeometry; check finite centres and derived bounds; radii must be >0.
Margin negative / NaN / +/-infinity	InvalidEllipseMargin(value); validate before any SVG creation.
Clock minute 60..255	ClockPositionError via TryFrom; no ClockPosition can represent it.
Clock on rectangle	InvalidEllipseEndpoint { endpoint, reason: "clock_on_rectangle" }.
Fraction on ellipse, even if no side	InvalidEllipseEndpoint { endpoint, reason: "fraction_on_ellipse" }.
Clock and forced side together	InvalidEllipseEndpoint { endpoint, reason: "clock_with_side" }.
Non-cardinal clock with elbow	InvalidEllipseElbowClock(clock), both when adding and switching type.

Validation order and atomicity

For edge creation validate the new connector type, both endpoint shapes and options, and the cardinal restriction **before** rendering or registering an edge. In set_connector_type, validate both stored clocks against the requested new type **before** changing the path/type. Errors must leave the edge unchanged.

Use a fixed set of reason strings as shown; no dynamically allocated error strings are required. Keep detailed documentation in the enum variants and Display implementations.

06 / Shape-aware endpoints and compatibility

A single shape-aware endpoint resolver must serve initial construction, move/drag rerouting and changing connector type. The two ends must be resolved independently.

Option	Rectangle end	Ellipse end
Auto straight	Existing continuous rectangle crossing	Continuous ellipse ray crossing, using that end's own bearing
Auto elbow	Existing first crossed side / midpoint	Nearest 00/15/30/45 by that end's own bearing
Forced side only	Unchanged forced-side rule	Exactly corresponding cardinal, **even for straight**
Forced side + fraction	Unchanged 0..=1 along side	InvalidEllipseEndpoint, even if fraction is otherwise valid
Explicit clock	InvalidEllipseEndpoint	All minutes for straight; cardinals only for elbow
Clock plus side	Not valid	InvalidEllipseEndpoint; no implicit priority rule

Mixed pairs are legal

- **Ellipse → rectangle:** resolve the source ellipse towards the target; resolve the target rectangle using its unchanged reverse-direction rule.
- **Rectangle → ellipse:** preserve the source rectangle rule; resolve the ellipse target towards the source.
- **Ellipse → ellipse:** each end independently uses its own (opposite) bearing; automatic elbow cardinals need not be the same.

The existing elbow router can connect arbitrary N/E/S/W end sides, including E out of an ellipse and N into a rectangle. Test all three pairs for straight and elbow routes, both initially and after movement.

No dynamic clock setter

The edge handle retains from_clock/to_clock as Option. These never change after add_edge_with in v1. set_connector_type is the only later setter needing to revalidate them. A future set_edge_clock would require its own validation, redraw and retry contract.

Off-cardinal elbow routes are not supported. They could need escape stubs beyond the current four-vertex Route and revised corner-radius handling. Never silently degrade to a cardinal or clip the ellipse.

07 / DOM, movement and interaction

Rendering and storage

- Draw actual <ellipse> using SvgRoot::ellipse(centre, Size::new(rx, ry)). Draw children in local coordinates; translate their containing <g> with full precision for moves.
- Store a shape discriminator and final radii alongside the graph node's enclosing Rect. The model must distinguish ellipse from rectangle; rerouting must never infer shape from the Rect alone.
- Use an outer SVG shape handle (for example BoxHandles::outer_shape) for focus-stroke changes. Keep the established role="group", aria-label and direct-child <title> pattern and the edge relationship descriptions.
- Retain RenderGuard rollback and established ownership/listener cleanup. No transparent rectangular hit target should be added.

Movement and coordinate convention

The add constructor takes a **centre**. move_node continues to take the **top-left corner of the final enclosing rectangle** for every node shape. Growth during construction preserves the centre. Movement translates the entire rendered group; final radii do not change during a move.

Hit testing, focus and background pan

Pointer interaction belongs to the visible ellipse fill and the label, not empty corners of the bounding Rect. With background panning enabled, a pointer-down-and-drag from such a corner **must pan the view**, while the ellipse **must neither move nor become focused**. Ordinary drags begun inside the painted ellipse must move it and reroute its edges.

Collision and edge labels

Existing rectangle bounding-box collision checks remain conservative. The edge-label placer likewise treats an ellipse as its enclosing Rect: labels may stand farther from its curved corners than strictly needed, but they never overlap the ellipse. The v1 design makes no claim of shape-tight packing.

Dijkstra demo and CDP fixture

When migrating Dijkstra vertices, update the CDP finder from a rect-plus-text assumption to a group with a title, ellipse and text. Identify the ellipse node semantically rather than by fragile child index. Prefer a circle radius of 22 or more for the focus ring.

08 / Failure recovery and resource budgets

Drawing failures and retry boundaries

- Node construction that fails during getBBox, SVG writing or attachment returns the error and removes partial DOM via RenderGuard. No node, edge or stray SVG handle is left registered.
- For any future operation that changes ellipse bounds, reroute incident edges and reposition affected edge labels; v1 performs fitting only during construction, before it has incident edges.
- The edge-label-change retry mechanism should be referenced as **the retry introduced by that change**, not asserted to have been committed in the checked main revision. No new independent stale-label state is proposed here.

Concrete security guidance (docs/security.md)

The repository currently defines **no numeric application limits**; it explicitly puts budget enforcement on applications accepting untrusted graph descriptions. V1 must preserve that policy, not invent default caps. Quote these actual budget categories in the ellipse docs:

- **String length:** limit plain/ellipse labels before getBBox; long text increases measured width, resulting geometry and stored accessible relationship strings.
- **Per-node degree:** cap the number of incident edges (routes) at any ellipse. Each move redraws every incident edge; pointer events may be coalesced, but work per animation frame still scales with degree.
- **Whole-scene budgets:** cap node/edge counts, count of draggable/interactively updated nodes, and optionally total DOM elements. The existing guidance also covers cell budgets for data nodes.

Reject or truncate untrusted descriptions **before** calling Scene. Finite geometry checks supplement application budgets; they do not replace them. Oversized labels or radii that overflow after fitting are InvalidEllipseGeometry, not silently clamped.

Numeric stability

Use explicit absolute/relative tolerances in all trigonometric comparisons. For a zero-length centre-to-centre vector, use the defined ellipse east fallback instead of normalising it. Validate the output is finite after each geometry step.

Checked reference: github.com/ChrisWhealy/svg-dom-graph/blob/main/docs/security.md, sections Node Degree, String Lengths, Suggested Budgets (23 October 2026).

09 / Phase 1: DOM-free native tests

Run these without a browser under cargo test. Both directions, floating-point ties and edge cases must be visible in fixtures. Geometry helpers must not depend on scene or SVG DOM handles.

Test	Required assertion
Radii constructors	circle(r) stores equal radii; new(rx,ry) returns each supplied radius. Invalid radii are rejected at scene add time, not constructor time.
Clock conversion	0..=59 succeed; 60 and 255 return ClockPositionError carrying the supplied value; cardinal constants equal 0/15/30/45.
Cardinal positions	P(0/15/30/45) equals N/E/S/W within tolerance for circle and noncircle.
Every minute	Each 0..59 P(m) lies on boundary and has required ray bearing; test 1, 10, 59 explicitly.
Straight auto	Non-cardinal centre ray intersects actual ellipse boundary continuously, not cardinal snap.
Elbow quantisation	Wide rx=100 ry=40: 30/40 degrees above E => E; 50/60 => N; exact 45-degree tie resolves clockwise.
Reverse target bearing	from and to use opposite centre vectors; source/target cardinals and points are independently correct.
Coincident centres	Two ellipses with same centre fall back to east at both ends; finite route points, no division by zero.
Label containment	80x16, circle r30, M6 => r~48.1; ellipse 70x32 is unchanged; preserve ratio after growth.
Zero metrics	w=0; h=0; empty 0x0 with M=6 and zero margin: no NaN, radii stay strictly positive.
Invalid geometry	Negative, 0, NaN, infinite radii/centres, overflows and invalid margin return precise graph error.

Tolerance comparator: abs(actual-expected) <= eps_abs + eps_rel * max(abs(actual),abs(expected)). Document chosen eps values in test helpers; do not compare trigonometric coordinates using ==.

10 / DOM, routing and CDP regression tests

After Phase 1, use browser tests for DOM measurement, endpoint dispatch, focus, movement, errors and accessibility.

Fixture	Observable result
Three mixed pairs: ellipse→rect, rect→ellipse, ellipse→ellipse	For both straight and elbow, each end resolves from its own shape and reverse/forward bearing. Check initial and moved paths.
Mismatched elbow sides	E out of ellipse can connect to N on rectangle; no assertion that the sides must match.
Forced-side straight	Force ellipse North: lands at 00, not at the automatic ray intersection; verify same after move.
Explicit off-cardinal straight	Set m=10 at source or target; endpoint lies on ellipse equation and stays attached after drag.
Invalid combinations / transition	Wrong-shape clock, ellipse fraction, clock plus side and non-cardinal elbow fail before mutation. Straight-to-elbow failure preserves path/type.
Label measurement / focus	Actual getBBox used; ellipse expanded only as needed, true shape gets focus stroke; accessibility group/title/relationships intact.
Unpainted-corner pan	With background pan enabled, pointer-down-and-drag from corner outside curve pans view; ellipse stays put and unfocused.
Painted ellipse drag	Pointer drag from fill moves node, reroutes all incident edges; centre/radii preserved.
Empty labels / browser metrics	No NaN, no zero-sized ellipse, including zero-width and 0x0 measurements.
Dijkstra demo selector	Find a vertex group with title + ellipse + text; no rect-only finder or index-based selector.
Edge-label clearance	Bounding-Rect proxy leaves conservative clearance, never overlaps painted ellipse.
Regression and failure	All rectangle-only routes, side fractions and EdgeAnchors unchanged; DOM error rolls back partial construction.

Do not derive expected route locations merely by reading the route that the code under test just generated. Use independently computed ellipse points or native fixture results.

11 / Delivery phases and release gate

1. **DOM-free geometry and clock**
   ClockPosition with ClockPositionError; radii value type; ellipse ray intersection; P(m); reverse bearings; quantisation and ties; coincident-centre fallback; pure fit function and native tests. No DOM changes.

2. **Ellipse construction and model**
   add_ellipse_node(_with), margin and geometry validation, real SVG ellipse, getBBox fit, authoritative shape storage, Rollback/RenderGuard and coordinate documentation.

3. **Shape-aware connectors**
   Independent source/target resolution; mixed pairs; forced-side straight override; clocks in ConnectorHandle; add-edge validation and set_connector_type atomic refusal.

4. **Interaction, demo and docs**
   Drag, focus, corner-pan test, edge-label conservative clearance, accessibility, Dijkstra fixture selectors, radius 22+ demo, exhaustive rectangle regressions and security documentation.

Release acceptance criteria

- Circle remains circular through label growth. All minute-0..59 explicit straight ports land on the true curved outline within numerical tolerance.
- Automatic straights are continuous; elbows use end-local angular cardinal choice with reverse target bearing. All three mixed-shape pairings pass.
- Forced-side straight ports are cardinal; non-cardinal elbows and incompatible endpoint configurations fail with named errors, without half-written edges.
- add_ellipse_node is centre-based; move_node and node_rect are final-Rect/top-left-based. Empty text and invalid inputs do not generate NaN or orphaned DOM.
- CDP covers background pan from bounding-box corners, filled-shape drag, focus, labels, accessibility, Dijkstra selectors and unchanged rectangle behaviour.

Reference files inspected

svg-dom-graph: src/scene/node/{plain,node_options,edge_anchors}.rs, src/scene/connector/{mod,connector_options,connector_handle}.rs, src/scene/scene_inner.rs, src/scene/box_handles/mod.rs, src/model/node/mod.rs, src/error/mod.rs, docs/library/{nodes,connectors,labels-and-text}.md, docs/security.md. svg-dom: src/root/ellipse.rs.

> **Implementation hand-off:** Phase 1 can start immediately. It is a native-only geometry + type change, with the accepted formulas, error boundaries and deterministic test expectations fixed above.